

Exercise 1: K-Means Clustering (Implementation Task)

Write a Python program to perform K-Means clustering.

Requirements:

- Use `make_blobs` to generate dataset:
 - `n_samples=300`
 - `centers=3`
 - `random_state=42`
- Apply K-Means:
 - `n_clusters=3`
- Extract:
 - Cluster labels
 - Cluster centers
- Plot:
 - Data points with cluster colors
 - Centroids using red X
- Save the plot as **kmeans_output.png**

Exercise 2: Elbow Method Implementation

Implement Elbow Method to determine optimal clusters.

Requirements:

1. Use same dataset
2. Run loop for $k = 1$ to 10
3. Store inertia values in a list
4. Plot graph:
 - X-axis $\rightarrow k$
 - Y-axis $\rightarrow$ inertia
5. Save graph as **elbow_plot.png**

Exercise 3: K-Medoids Clustering (PAM Algorithm)

Apply K-Medoids clustering and visualize results.

Requirements:

Install library:

`pip install scikit-learn-extra`

- Generate dataset:
 - `n_samples=300`
 - `centers=3`
- Apply K-Medoids:
 - `n_clusters=3`
 - `method='pam'`
- Extract:
 - Medoids

- Labels
- Plot:
 - Clustered data
 - Medoids using red X
- Save output as **kmedoids_output.png**

Exercise 4: K-Means Implementation

Task

Implement the **K-Means clustering algorithm from scratch** (without using `sklearn.cluster.KMeans`).

Requirements:

- Generate dataset:
 - from `sklearn.datasets` import `make_blobs`
 - `X, _ = make_blobs(n_samples=200, centers=3, random_state=42)`
- Initialize:
 - Choose $k = 3$
 - Randomly select 3 points from dataset as initial centroids
 - -----
- Implement the following steps manually:
 - ✓ Assign each point to nearest centroid
 - ✓ Update centroids by taking mean of assigned points
 - ✓ Repeat until:
 - Centroids do not change **OR**
 - Maximum iterations (e.g., 100) reached
 - -----
- Store:
 - Final centroids
 - Cluster labels
 - -----
- Plot results:
 - Data points with cluster colors
 - Final centroids using red X
 - -----
- Save output as:

- kmeans_scratch.png